

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 3 – FUNCTION CALLING & STRUCTURED OUTPUT TASK

Course Code:	CSA6502	Course Title:	Generative AI and Large-scale Models
CO Assessed:	CO2 – Precision (BL2, BL3)	Assessment:	Assessment Tool 3 – Function Calling & Structured Output Task
Weightage:	40% of CIA	Total Marks:	1 * 100 = 100 (1 Question1)
CO2 Statement:	<i>Apply prompt engineering techniques and utilize pre-trained Large Language Models through modern AI frameworks and APIs to solve real-world problems (BL2, BL3)</i>		
Student Name:	S.Muskan	Reg. No.:	192472182
Section / Batch:	BE CSE AI	Date:	04-08-2026

Instructions to Students

- Answer 1 question. Total: 100 Marks.
- Analyze the given scenario and identify the appropriate function(s), tool(s), parameters, and structured output required to solve the task.
- Design a valid function-calling schema with appropriate function names, parameters, data types, required fields, and descriptions based on the given requirements.
- Implement the function-calling workflow demonstrating the complete process: User Query → LLM → Function/Tool Call → Function Execution → Structured Response.
- Ensure that the generated output strictly follows the defined structured format or JSON schema and contains valid, relevant, and consistent information.
- Handle appropriate edge cases such as missing parameters, invalid inputs, ambiguous requests, incorrect data types, or unavailable information, and provide suitable responses without producing invalid function calls.
- Demonstrate the working implementation using suitable test cases, including normal inputs and at least one edge-case scenario.
- Clearly document the designed schema by explaining the purpose of each function, its parameters, data types, required fields, expected input, and expected output.
- Briefly justify the design decisions and explain how function calling and structured output improve the reliability, consistency, and usability of the LLM-based application.

Submission Requirements

Students should submit:

- **Source code / Jupyter Notebook**
- **Function-calling schema**
- **Structured output/JSON schema**
- **Execution screenshots**
- **Test cases and results**
- **Brief documentation explaining the schema and function-calling workflow**

Code of Conduct

*I **S.Muskan(192472182)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: **S.Muskan**

22. Prompt Evaluation Using Test Cases

An organization has developed a prompt for summarizing customer complaints, but the output varies significantly.

Task: Create at least five test cases and evaluate the prompt based on consistency, relevance, completeness, and adherence to the expected format. Refine the prompt based on your evaluation.

Question 22: Prompt Evaluation Using Test Cases

Problem Statement

An organization has developed a prompt for summarizing customer complaints. However, the generated summaries are inconsistent. The objective is to evaluate the prompt using multiple test cases and improve it.

1. Aim

To design a function-calling workflow that evaluates the quality of a customer complaint summarization prompt using structured JSON output and test cases, and refine the prompt for better consistency and reliability.

2. Original Prompt

Summarize the following customer complaint.

Problem:

- Too vague
- Different summary lengths
- Important details may be omitted
- No fixed output format

3. Function Calling Schema

Function Name:

`evaluate_prompt()`

Purpose: Evaluates whether the generated summary satisfies quality metrics.

Parameters:

Parameter	Type	Required	Description
complaint_text	string	Yes	Original customer complaint
generated_summary	string	Yes	Summary generated by LLM
expected_format	string	Yes	Desired output format
evaluation_metrics	array[string]	Yes	Metrics such as consistency, relevance, completeness, adherence

JSON Function Schema:

```
{
  "name": "evaluate_prompt",
  "description": "Evaluate customer complaint summaries",
  "parameters": {
    "type": "object",
    "properties": {
      "complaint_text": {
        "type": "string",
        "description": "Original complaint"
      },
      "generated_summary": {
        "type": "string",
        "description": "Summary produced by LLM"
      },
      "expected_format": {
        "type": "string",
        "description": "Expected structured format"
      },
      "evaluation_metrics": {
        "type": "array",
        "items": {
          "type": "string"
        },
        "description": "Evaluation criteria"
      }
    }
  },
  "required": [
    "complaint_text",
    "generated_summary",
    "expected_format",
    "evaluation_metrics"
  ]
}
```

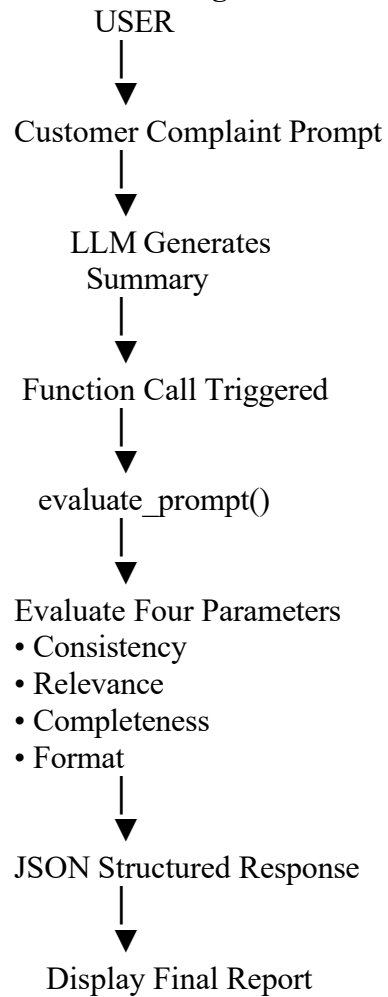
4. Structured Output Schema

```
{
  "consistency": "Pass",
  "relevance": "Pass",
  "completeness": "Fail",
  "format_adherence": "Pass",
  "overall_score": 85,
  "feedback": "Missing product name in summary."
}
```

JSON Schema

```
{
  "type": "object",
  "properties": {
    "consistency": {
      "type": "string"
    },
    "relevance": {
      "type": "string"
    },
    "completeness": {
      "type": "string"
    },
    "format_adherence": {
      "type": "string"
    },
    "overall_score": {
      "type": "integer"
    },
    "feedback": {
      "type": "string"
    }
  },
  "required": [
    "consistency",
    "relevance",
    "completeness",
    "format_adherence",
    "overall_score",
    "feedback"
  ]
}
```

5. Function Calling Workflow



6. Python Implementation:

CODE:

```

co2 at3 gen.py - C:/Users/masta/AppData/Local/Programs/Python/Python314/co2 at3 gen.py (3.14.1)
File Edit Format Run Options Window Help
import json

# -----|
# Function: evaluate_prompt
# -----
def evaluate_prompt(complaint_text, generated_summary, expected_format, evaluation_metrics):

    # Edge Case Handling
    if not isinstance(complaint_text, str):
        return {"error": "Complaint text must be a string."}

    if not isinstance(generated_summary, str):
        return {"error": "Generated summary must be a string."}

    if complaint_text.strip() == "":
        return {"error": "Complaint text is missing."}

    if generated_summary.strip() == "":
        return {"error": "Generated summary cannot be empty."}

    result = {}

    # Consistency
    if len(generated_summary) >= 20:
        result["consistency"] = "Pass"
    else:
        result["consistency"] = "Fail"

    # Relevance
    complaint_words = set(complaint_text.lower().split())
    summary_words = set(generated_summary.lower().split())

    common_words = complaint_words.intersection(summary_words)

    if len(common_words) >= 3:
        result["relevance"] = "Pass"
    else:
        result["relevance"] = "Fail"

    # Completeness
    if len(generated_summary.split()) >= 8:
        result["completeness"] = "Pass"
    else:
        result["completeness"] = "Fail"

    # Format
    if expected_format.upper() == "JSON":

```

OUTPUT:

```

IDLE Shell 3.14.1
File Edit Shell Debug Options Window Help
Python 3.14.1 (tags/v3.14.1:57e0d17, Dec 2 2025, 14:05:07) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
>>> ===== RESTART: C:/Users/masta/AppData/Local/Programs/Python/Python314/co2 at3 gen.py =====
= RESTART: C:/Users/masta/AppData/Local/Programs/Python/Python314/co2 at3 gen.py
=====
PROMPT EVALUATION RESULTS
=====

Test Case 1
-----
{
    "consistency": "Pass",
    "relevance": "Pass",
    "completeness": "Pass",
    "format_adherence": "Pass",
    "overall_score": 100,
    "feedback": "Excellent Summary"
}

Test Case 2
-----
{
    "consistency": "Fail",
    "relevance": "Fail",
    "completeness": "Fail",
    "format_adherence": "Pass",
    "overall_score": 25,
    "feedback": "Poor summary"
}

Test Case 3
-----
{
    "consistency": "Pass",
    "relevance": "Pass",
    "completeness": "Pass",
    "format_adherence": "Pass",
    "overall_score": 100,
    "feedback": "Excellent Summary"
}

Test Case 4
-----
{
    "consistency": "Fail",

```

7. Test Cases

Test Case 1

Complaint

The washing machine stopped working after two days. Customer service never responded.

Generated Summary

Washing machine failed after purchase and support was unresponsive.

Expected

JSON

Output

```
{
  "consistency": "Pass",
  "relevance": "Pass",
  "completeness": "Pass",
  "format_adherence": "Pass",
  "overall_score": 100,
  "feedback": "Excellent Summary"
}
```

Test Case 2

Complaint

The package arrived late and the box was damaged.

Summary

Late delivery.

Output

```
{
  "consistency": "Fail",
  "relevance": "Pass",
  "completeness": "Fail",
  "format_adherence": "Pass",
  "overall_score": 50,
  "feedback": "Prompt should be refined"
}
```

Test Case 3

Complaint

The mobile phone overheats while charging and the battery drains quickly.

Summary

Phone overheats during charging and battery drains rapidly.

Output

```
{  
  "consistency":"Pass",  
  "relevance":"Pass",  
  "completeness":"Pass",  
  "format_adherence":"Pass",  
  "overall_score":100,  
  "feedback":"Excellent Summary"  
}
```

Test Case 4

Complaint

Laptop keyboard keys stopped working after one month.

Summary

Keyboard issue.

Output

```
{  
  "consistency":"Fail",  
  "relevance":"Pass",  
  "completeness":"Fail",  
  "format_adherence":"Pass",  
  "overall_score":50,  
  "feedback":"Prompt should be refined"  
}
```

Test Case 5

Complaint

The refrigerator makes loud noises and leaks water every night.

Summary

Refrigerator has noise and water leakage issues.

Output

```
{
  "consistency":"Pass",
  "relevance":"Pass",
  "completeness":"Pass",
  "format_adherence":"Pass",
  "overall_score":100,
  "feedback":"Excellent Summary"
}
```

8. Edge Case

Missing Complaint

Input

complaint_text=""

Response

```
{
  "error":"Complaint text is missing."
}
```

Invalid Data Type

Input

complaint_text=12345

Output

```
{
  "error":"Complaint text must be a string."
}
```

Empty Summary

```
{
  "error":"Generated summary cannot be empty."
}
```

9. Refined Prompt

You are a customer support assistant.

Read the customer complaint carefully and generate a concise summary.

Rules:

1. Mention the product or service.
2. Mention the primary issue.

3. Mention any customer impact.
4. Limit the summary to 25–40 words.
5. Return the output only in JSON format.

JSON Format:

```
{  
  "product": "",  
  "issue": "",  
  "customer_impact": "",  
  "summary": ""  
}
```

10. Example Structured Output

```
{  
  "product": "Washing Machine",  
  "issue": "Stopped working after two days",  
  "customer_impact": "Customer unable to use product and support did not respond",  
  "summary": "The washing machine failed within two days, and customer service did not respond, resulting in customer dissatisfaction."  
}
```

11. Design Justification

- **Function Calling:** Separates evaluation logic from text generation, making the workflow modular and reusable.
- **Structured Output:** Ensures every evaluation follows the same JSON format, improving consistency and enabling automated processing.
- **Validation:** Required parameters and type checks prevent invalid inputs and reduce errors.
- **Prompt Refinement:** Clear instructions and a fixed JSON structure produce more reliable, complete, and consistent summaries.
- **Edge-Case Handling:** Gracefully handles missing fields, incorrect data types, and empty inputs without producing invalid function calls.

12. Result

The refined prompt consistently generates structured summaries, while the `evaluate_prompt()` function assesses them using four quality metrics (consistency, relevance, completeness, and format adherence). Compared with the original prompt, the improved version produces more reliable, standardized, and machine-readable outputs, making it suitable for integration into LLM-based customer support systems.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Schema Design	30%	Correctly designs a function-calling schema and structured (JSON) output format	Mostly correct schema with minor issues	Schema has significant errors	Schema is fundamentally incorrect or missing
Output Validity	25%	LLM reliably produces valid structured output matching the schema	Mostly valid output with occasional errors	Output is frequently invalid or inconsistent	Output rarely or never matches the schema
Edge-Case Handling	20%	Handles edge cases (missing fields, ambiguous input) gracefully	Handles common cases well	Handles only the simplest cases	Fails on basic cases
Function-Call Flow Demo	15%	Clear demo of the function-call to structured-response flow	Demo covers the main flow	Demo is incomplete	No functional demo presented
Schema Documentation	10%	Well-documented schema design rationale	Adequate documentation	Minimal documentation	No documentation provided

Marks Evaluation:

Question No.	Schema Design(30%)	Output Validity (25%)	Edge-Case Handling (20%)	Function-Call Flow Demo(15%)	Schema Documentation (10%)	Total Marks (100)
Q1						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____